Spring Modulith : deux modules communiquant par événement

Objectif

Comprendre le monolithe modulaire avec Spring Modulith 2.0 sur un exemple minimal : deux modules, un événement, un listener. On ne refait pas d'hexagonal ici ; l'intérieur des modules est volontairement simple pour garder l'attention sur les concepts Modulith.

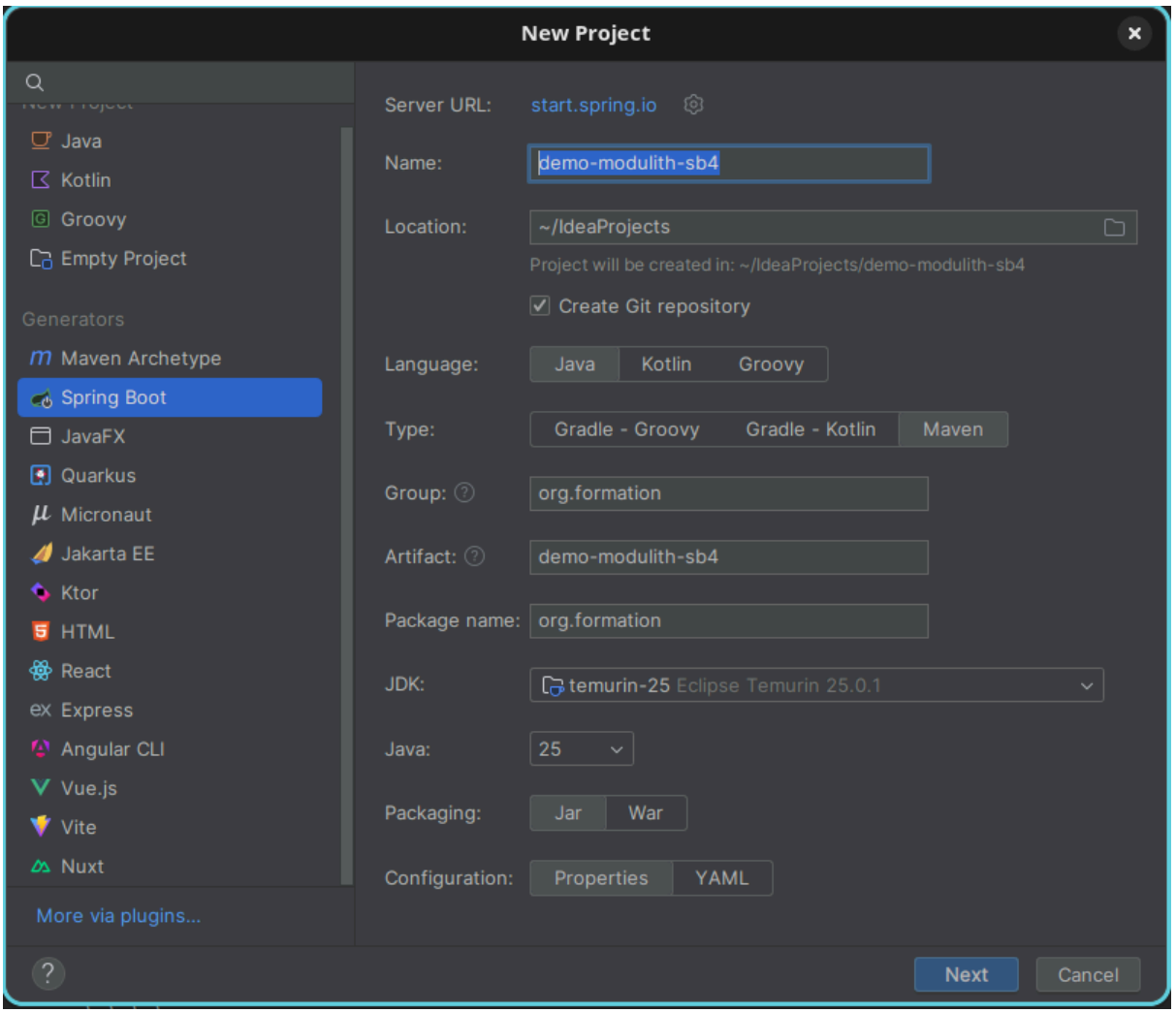
Les quatre idées à retenir

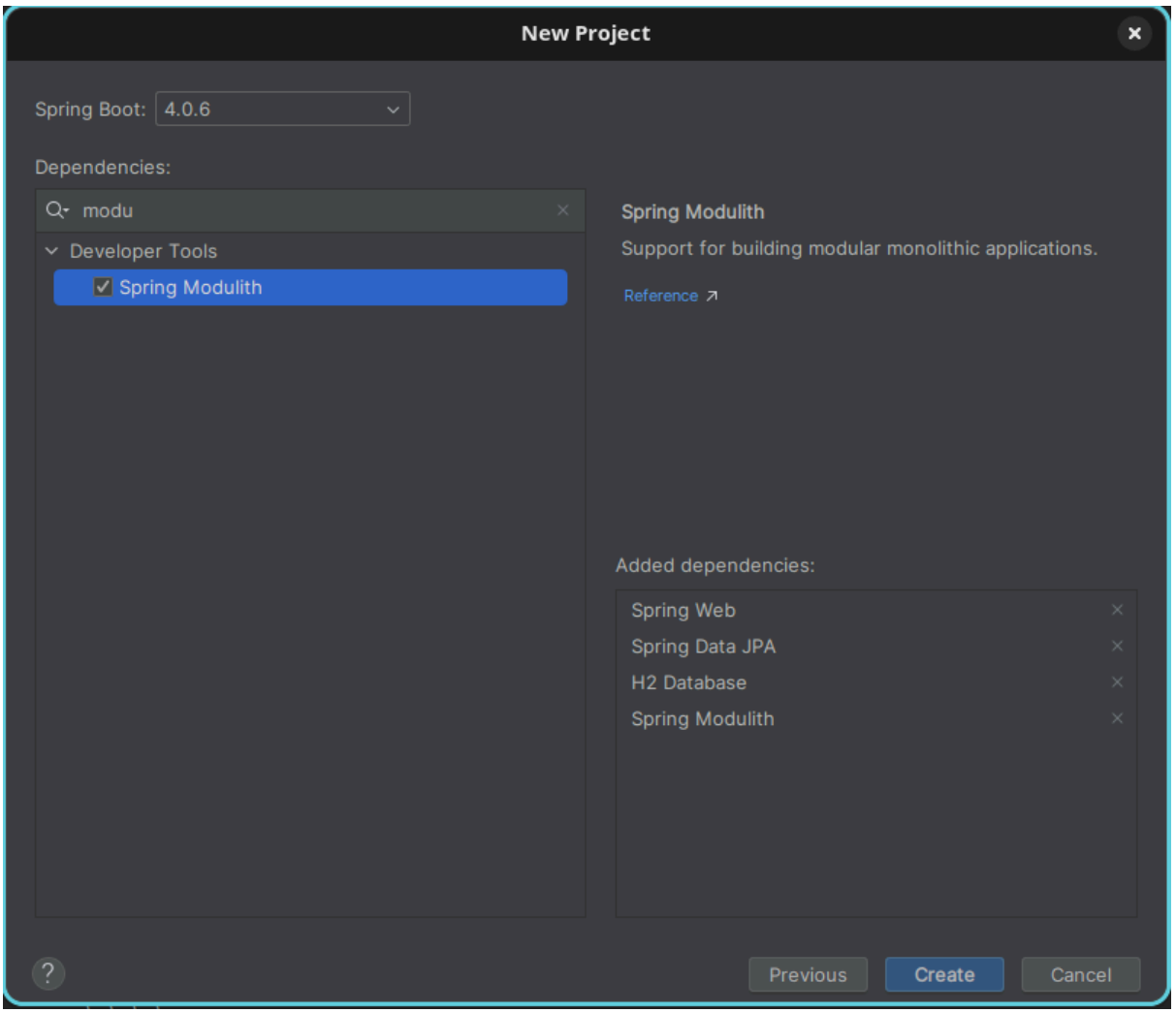
1. Un module est un sous-package direct de la racine, avec une API exposée et un `internal` caché.
2. Les modules communiquent par événement, jamais par appel direct.
3. `@ApplicationModuleListener` réagit à l'événement, après le commit, dans sa propre transaction.
4. L'Event Publication Registry persiste chaque publication (table JPA) et permet le rejeu.

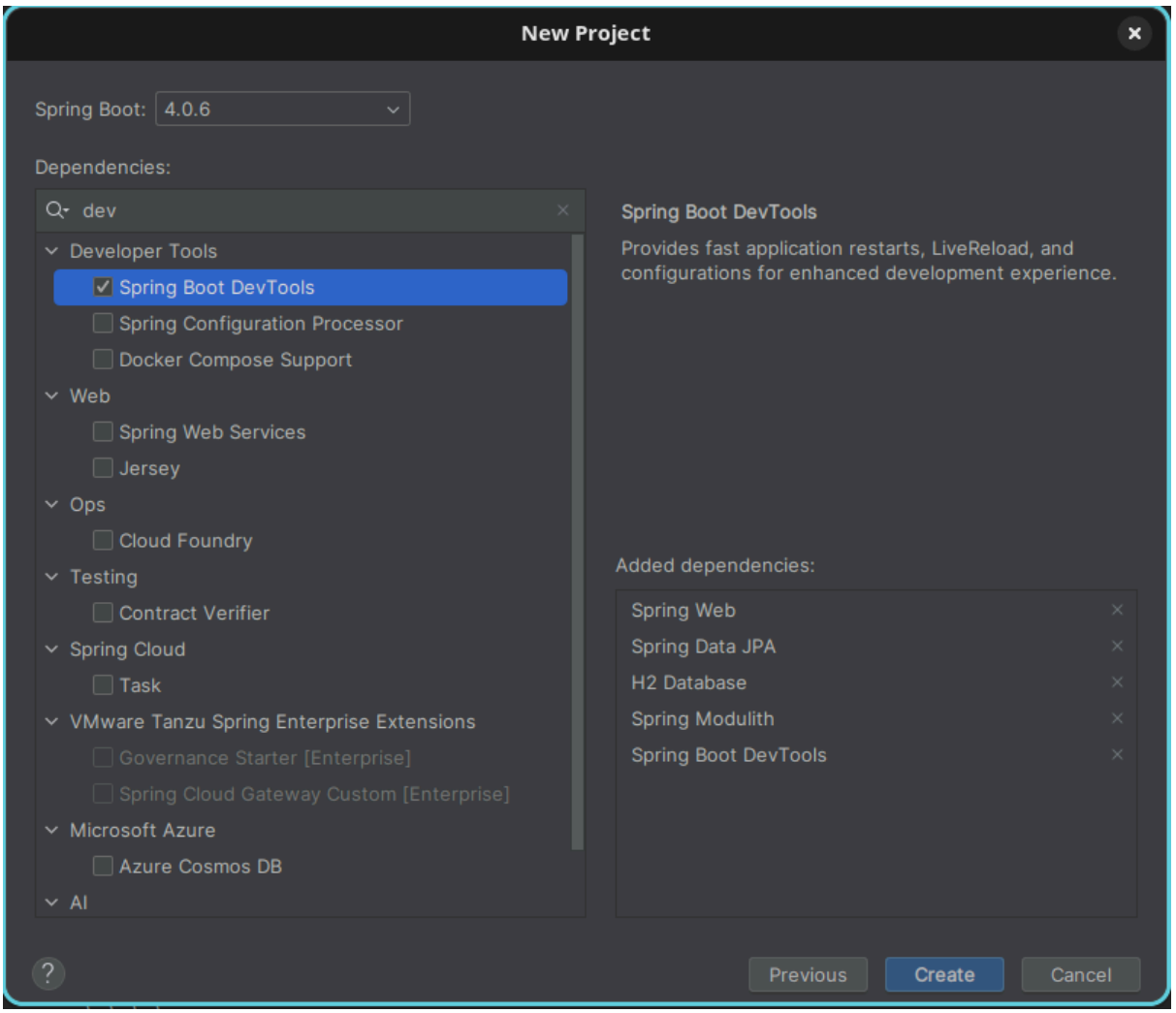
Étape 1 — Créer le projet et les dépendances

Projet : demo-modulith-sb4 Group et package : org.formation Maven Java 25

Sur Initializr (Spring Boot 4.0.x, Java 21, Maven), cocher : Spring Web, Spring Data JPA, H2 Database, et Spring Modulith. La case Spring Modulith ajoute le BOM, `spring-modulith-starter-core` et `spring-modulith-starter-test`.


New Project
Java
Kotlin
Groovy
Empty Project
Generators
Maven Archetype
Spring Boot
JavaFX
Quarkus
Micronaut
Jakarta EE
Ktor
HTML
React
Express
Angular CLI
Vue.js
Vite
Nuxt
More via plugins...
Server URL: start.spring.io
Name: demo-modulith-sb4
Location: ~/IdeaProjects
Project will be created in: ~/IdeaProjects/demo-modulith-sb4
Create Git repository
Language: Java Kotlin Groovy
Type: Gradle - Groovy Gradle - Kotlin Maven
Group: org.formation
Artifact: demo-modulith-sb4
Package name: org.formation
JDK: temurin-25 Eclipse Temurin 25.0.1
Java: 25
Packaging: Jar War
Configuration: Properties YAML
Next
Cancel

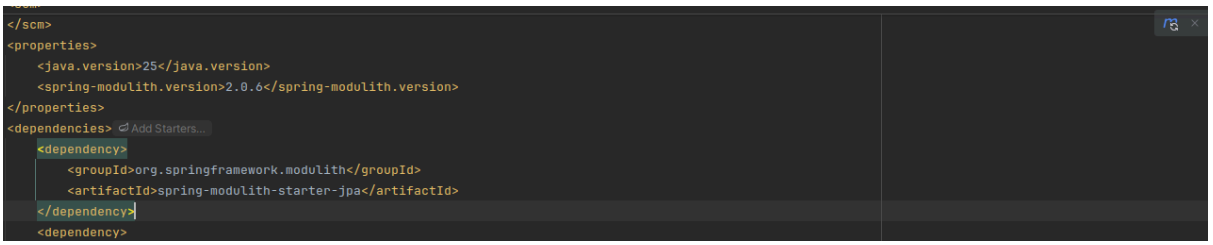

New Project
Spring Boot: 4.0.6
Dependencies:
modu
Developer Tools
Spring Modulith
Spring Modulith
Support for building modular monolithic applications.
Reference
Added dependencies:
Spring Web
Spring Data JPA
H2 Database
Spring Modulith
Previous
Create
Cancel



Ajouter aussi Lombok si vous voulez

Ajouter ensuite à la main le store du registre d'événements en JPA :

```
<dependency>
    <groupId>org.springframework.modulith</groupId>
    <artifactId>spring-modulith-starter-jpa</artifactId>
</dependency>
```



update maven

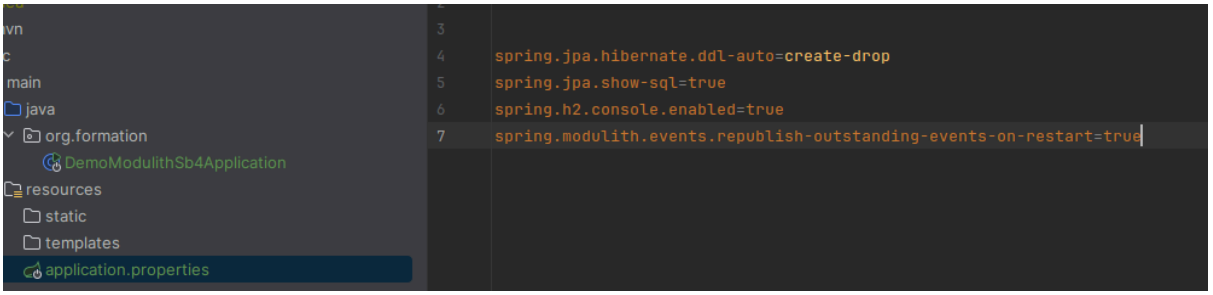
Vérifier que le BOM est bien présent (version 2.0.x, alignée sur Spring Boot 4) :

```
<dependencyManagement>
    <dependencies>
```

```
        <dependency>
            <groupId>org.springframework.modulith</groupId>
            <artifactId>spring-modulith-bom</artifactId>
            <version>2.0.0</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

```
    </dependencies>
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.modulith</groupId>
                <artifactId>spring-modulith-bom</artifactId>
                <version>${spring-modulith.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>
```

`application.properties` :



On garde `create-drop` : Hibernate crée le schéma, y compris la table `event_publication` du registre. En `validate`, le registre JPA poserait problème.

Étape 2 — La structure en deux modules

```
org.formation.shop
├── ShopApplication.java
├── catalog
│   ├── Catalog.java            (API : service public)
│   ├── CoffeeCreated.java      (API : evenement publie hors du module)
│   ├── CatalogController.java  (declencheur HTTP minimal)
│   └── internal
│       ├── CoffeeEntity.java   (cache)
│       └── CoffeeRepository.java
└── inventory
    └── internal
        ├── StockEntity.java
        ├── StockRepository.java
```

```
        └── InventoryListener.java
```

Règle clé : `catalog.internal` est invisible des autres modules. L'événement `CoffeeCreated`, lui, est dans le package racine du module `catalog` : il fait donc partie de son API et `inventory` a le droit de le référencer.

Étape 3 — Le module catalog

`catalog/internal/CoffeeEntity.java` :

pom.xml (demo-modulith-sb4) application.properties CoffeeEntity.java

```java
package org.formation.catalog.internal;


import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity  new *
public class CoffeeEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;  2 usages
    private int price;  1 usage

    protected CoffeeEntity() {  new *
    }

    public CoffeeEntity(String name, int price) {  no usages  new
        this.name = name;
        this.price = price;
    }

    public Long getId() {  new *
        return id;
    }

    public String getName() {  no usages  new *
        return name;
    }
}
```

`catalog/internal/CoffeeRepository.java` :

```java
package org.formation.catalog.internal;

import org.springframework.data.jpa.repository.JpaRepository;

public interface CoffeeRepository extends JpaRepository<CoffeeEntity, Long> {
}
```

`catalog/CoffeeCreated.java` (l'événement, partie de l'API) :

```java
package org.formation.catalog;

public record CoffeeCreated(Long coffeeId, String name) {
}
```

`catalog/Catalog.java` (le service public) :

```java
package org.formation.catalog;

import lombok.RequiredArgsConstructor;
import org.formation.catalog.internal.CoffeeEntity;
import org.formation.catalog.internal.CoffeeRepository;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@RequiredArgsConstructor
public class Catalog {

    private final CoffeeRepository repository;
    private final ApplicationEventPublisher events;


    @Transactional
    public Long createCoffee(String name, int price) {
        CoffeeEntity saved = repository.save(new CoffeeEntity(name, price));
        events.publishEvent(new CoffeeCreated(saved.getId(), saved.getName()));
        return saved.getId();
    }
}
```

Point central : `catalog` publie un événement et ne connaît pas `inventory`. La publication a lieu dans la transaction de `createCoffee`, ce qui permet au registre d'enregistrer la publication de façon transactionnelle.

`catalog/CatalogController.java` (déclencheur minimal) :

```java
package org.formation.catalog;

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController  new *
@RequestMapping("coffees")
class CatalogController {

    private final Catalog catalog;  2 usages

    CatalogController(Catalog catalog) {  new *
        this.catalog = catalog;
    }

    @PostMapping  new *
    Long create(@RequestParam String name, @RequestParam int price) {
        return catalog.createCoffee(name, price);
    }
}
```

Étape 4 — Le module inventory

`inventory/internal/StockEntity.java` :

```java
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity  new *
public class StockEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Long coffeeId;  2 usages
    private int quantity;  1 usage

    protected StockEntity() {  new *
    }

    public StockEntity(Long coffeeId, int quantity) {  no usages  new *
        this.coffeeId = coffeeId;
        this.quantity = quantity;
    }

    public Long getCoffeeId() {  no usages  new *
        return coffeeId;
    }
}
```

`inventory/internal/StockRepository.java` :

```java
package org.formation.inventory.internal;


import org.springframework.data.jpa.repository.JpaRepository;

public interface StockRepository extends JpaRepository<StockEntity, Long> {  no usages  new *
}
```

`inventory/internal/InventoryListener.java` :

```
package org.formation.shop.inventory.internal;
```

```java
package org.formation.inventory.internal;
import org.formation.catalog.CoffeeCreated;
import org.springframework.modulith.events.ApplicationModuleListener;
import org.springframework.stereotype.Component;

@Component  new *
class InventoryListener {

    private final StockRepository repository;  2 usages

    InventoryListener(StockRepository repository) {  new *
        this.repository = repository;
    }

    @ApplicationModuleListener  new *
    void on(CoffeeCreated event) {
        repository.save(new StockEntity(event.coffeeId(),  quantity: 0));
    }
}
```

Explication : `@ApplicationModuleListener` combine `@TransactionalEventListener` (déclenché après le commit de la transaction de publication), `@Transactional(REQUIRES_NEW)` (le listener s'exécute dans sa propre transaction) et `@Async`. Sans `@EnableAsync` dans le projet, la partie `@Async` reste dormante et le listener s'exécute de façon synchrone après le commit : c'est plus simple à

observer pour ce premier contact. Ajouter `@EnableAsync` rend la livraison asynchrone, ce qui est le défaut recommandé en production.

Étape 5 — Déclencher et observer

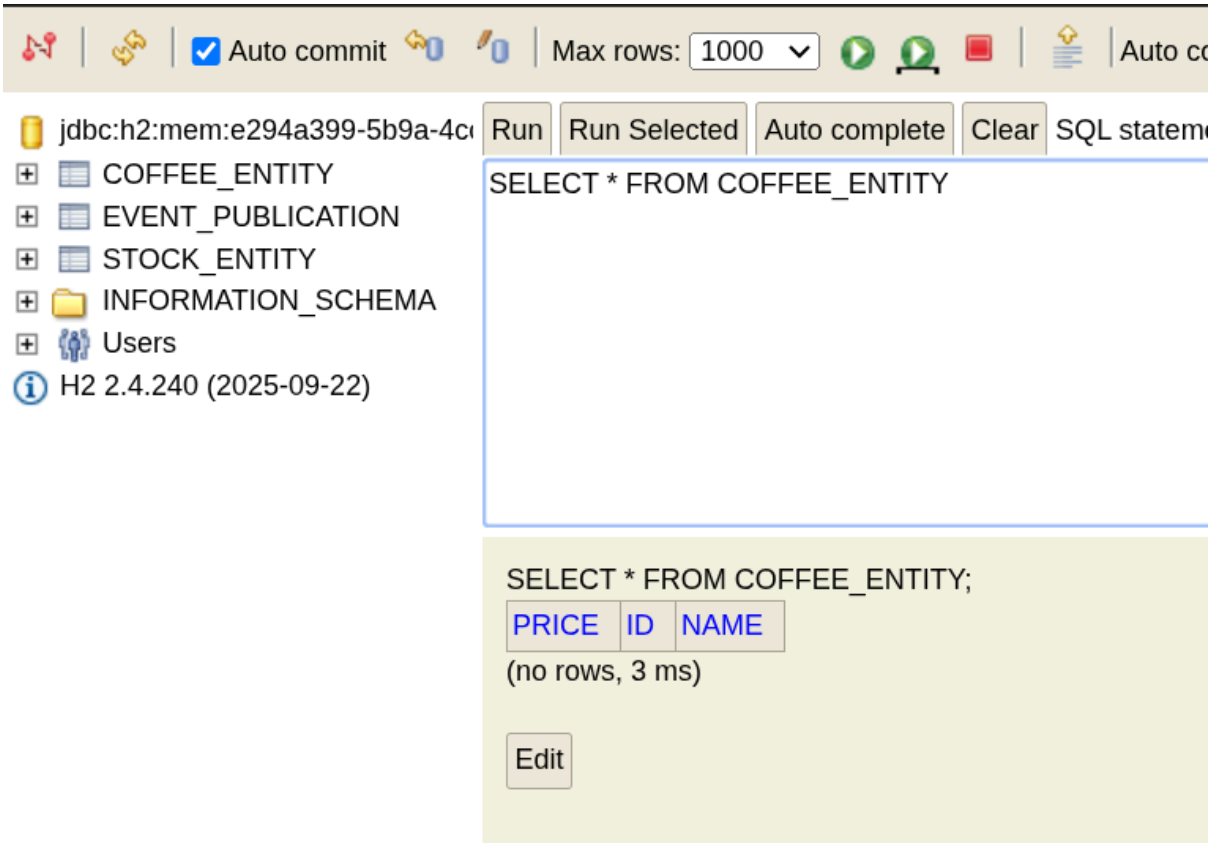
Démarrer l'application et créer un café :

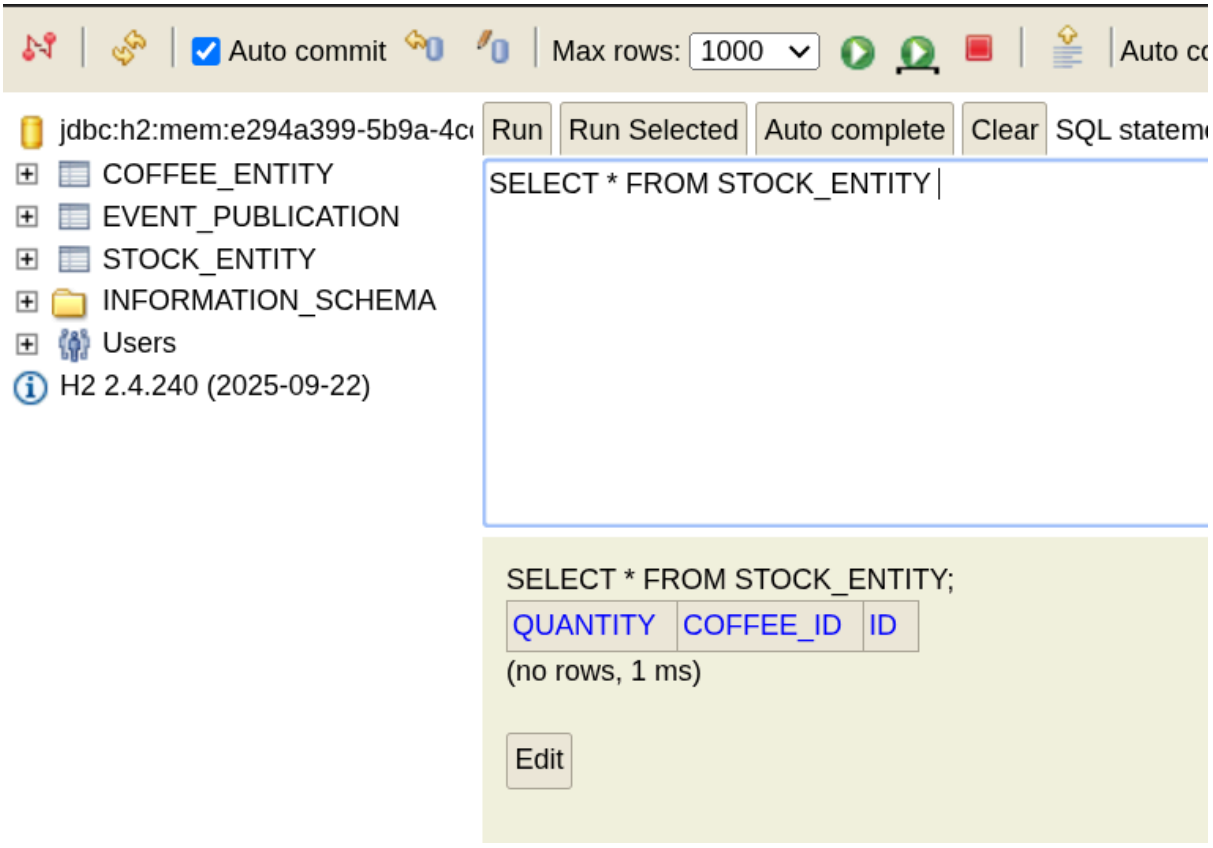
```
curl -X POST "http://localhost:8080/coffees?name=Espresso&price=10"
```

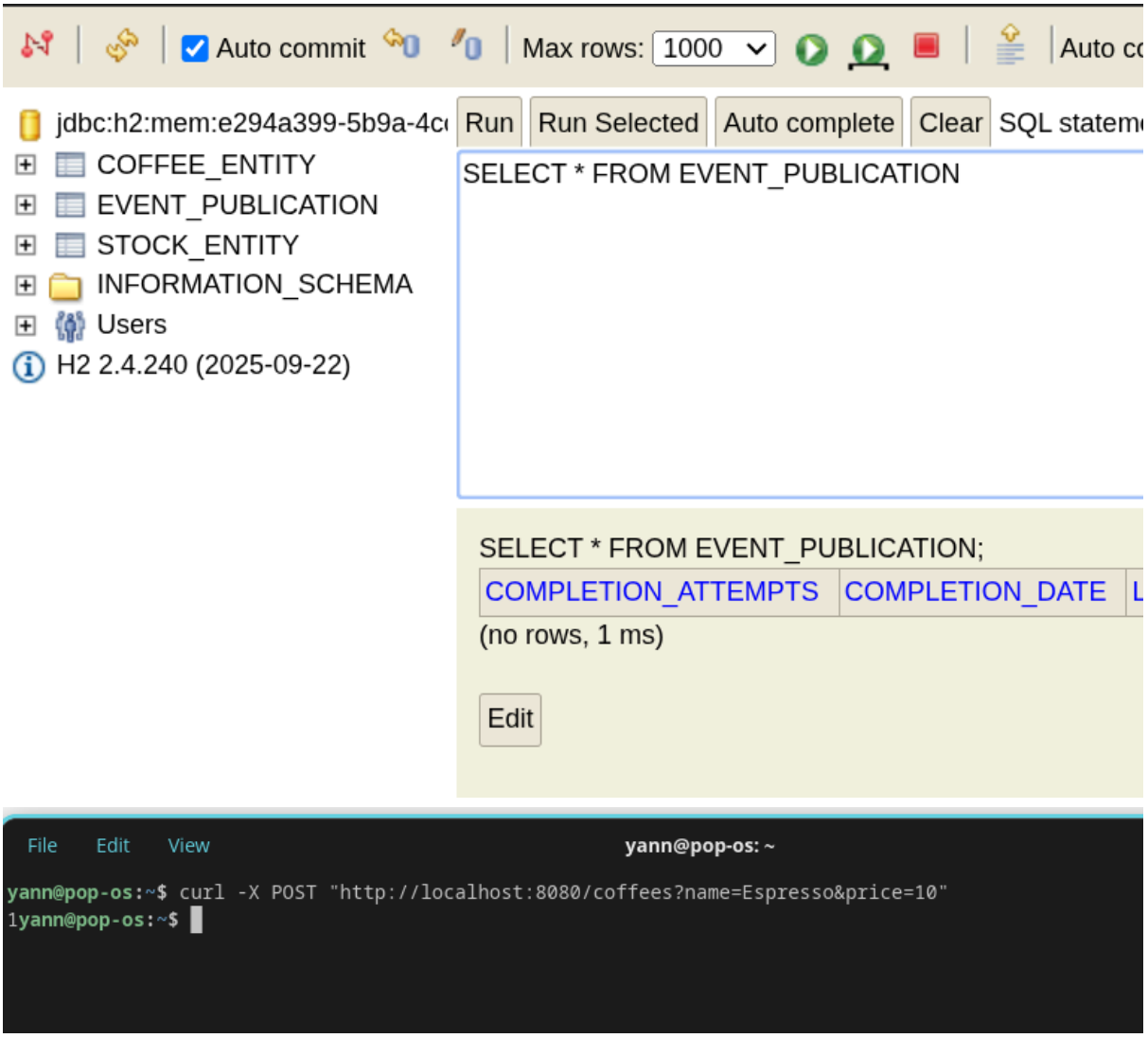
Ouvrir la console H2 (http://localhost:8080/h2-console) et vérifier :

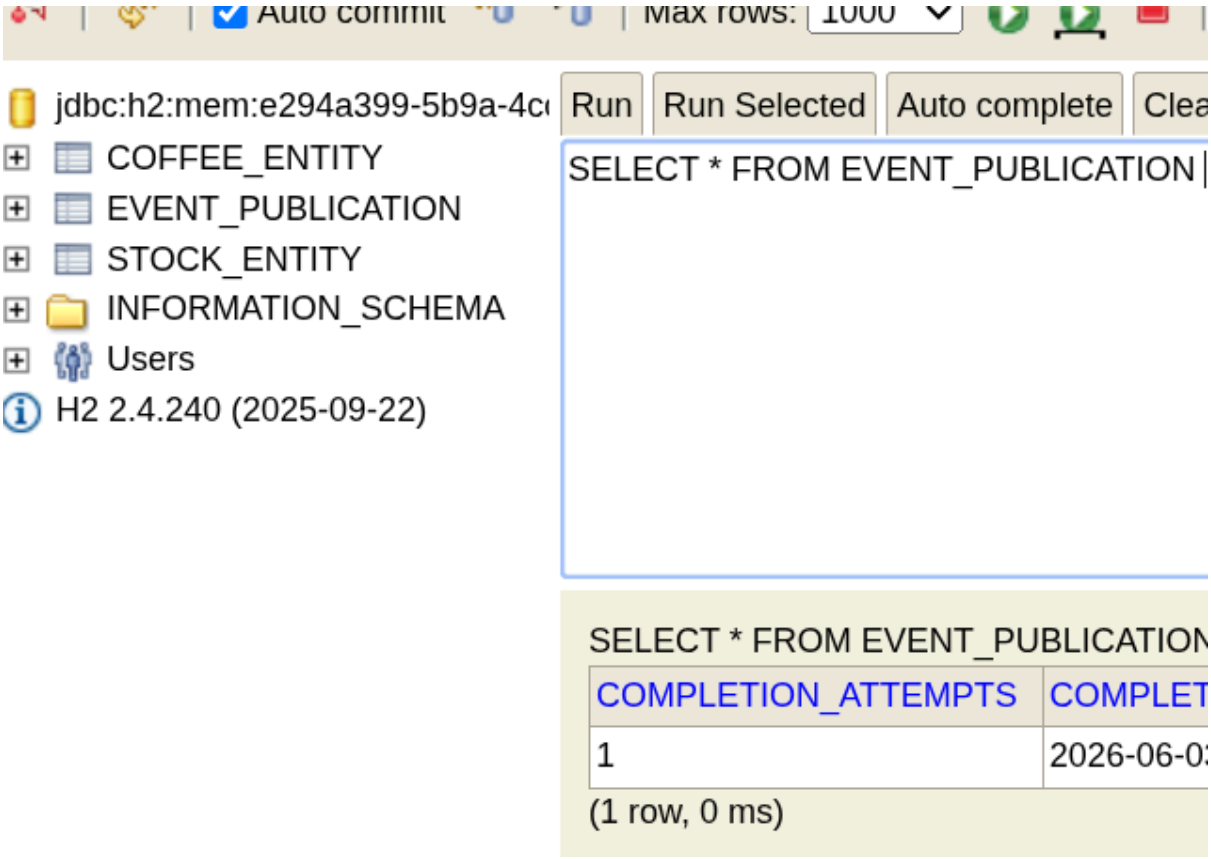
- `select * from coffee_entity;` contient l'espresso.
- `select * from stock_entity;` contient une ligne de stock créée par `inventory`, alors que `catalog` ne l'a jamais appelé.
- `select * from event_publication;` montre la trace de la publication enregistrée par le registre.

Le point important : le stock est apparu par le seul jeu de l'événement, sans dépendance directe entre les modules.


Auto commit
Max rows: 1000
Auto co
jdbc:h2:mem:e294a399-5b9a-4c
Run
Run Selected
Auto complete
Clear
SQL statem
COFFEE_ENTITY
EVENT_PUBLICATION
STOCK_ENTITY
INFORMATION_SCHEMA
Users
H2 2.4.240 (2025-09-22)
SELECT * FROM COFFEE_ENTITY
SELECT * FROM COFFEE_ENTITY;
PRICE
ID
NAME
(no rows, 3 ms)
Edit


Auto commit
Max rows: 1000
Auto co
jdbc:h2:mem:e294a399-5b9a-4c
Run
Run Selected
Auto complete
Clear
SQL statem
COFFEE_ENTITY
EVENT_PUBLICATION
STOCK_ENTITY
INFORMATION_SCHEMA
Users
H2 2.4.240 (2025-09-22)
SELECT * FROM STOCK_ENTITY
SELECT * FROM STOCK_ENTITY;
QUANTITY
COFFEE_ID
ID
(no rows, 1 ms)
Edit


Auto commit
Max rows: 1000
Auto co
jdbc:h2:mem:e294a399-5b9a-4c
Run
Run Selected
Auto complete
Clear
SQL statem
COFFEE_ENTITY
EVENT_PUBLICATION
STOCK_ENTITY
INFORMATION_SCHEMA
Users
H2 2.4.240 (2025-09-22)
SELECT * FROM EVENT_PUBLICATION
SELECT * FROM EVENT_PUBLICATION;
COMPLETION_ATTEMPTS
COMPLETION_DATE
(no rows, 1 ms)
Edit
File
Edit
View
yann@pop-os: ~
yann@pop-os:~$ curl -X POST "http://localhost:8080/coffees?name=Espresso&price=10"
1yann@pop-os:~$


Auto commit
Max rows: 1000
jdbc:h2:mem:e294a399-5b9a-4c
COFFEE_ENTITY
EVENT_PUBLICATION
STOCK_ENTITY
INFORMATION_SCHEMA
Users
H2 2.4.240 (2025-09-22)
Run
Run Selected
Auto complete
Clea
SELECT * FROM EVENT_PUBLICATION
SELECT * FROM EVENT_PUBLICATION
COMPLETION_ATTEMPTS
COMPLET
1
2026-06-0
(1 row, 0 ms)



jdbc:h2:mem:e294a399-5b9a-4c

- COFFEE_ENTITY
- EVENT_PUBLICATION
- STOCK_ENTITY
- INFORMATION_SCHEMA
- Users

H2 2.4.240 (2025-09-22)

Run | Run Selected | Auto complete | Clea

```
SELECT * FROM STOCK_ENTITY
```

SELECT * FROM STOCK_ENTITY;

QUANTITY	COFFEE_ID	ID
0	1	1

(1 row, 1 ms)



jdbc:h2:mem:e294a399-5b9a-4c

- COFFEE_ENTITY
- EVENT_PUBLICATION
- STOCK_ENTITY
- INFORMATION_SCHEMA
- Users

H2 2.4.240 (2025-09-22)

Run | Run Selected | Auto complete | Clea

```
SELECT * FROM COFFEE_ENTITY
```

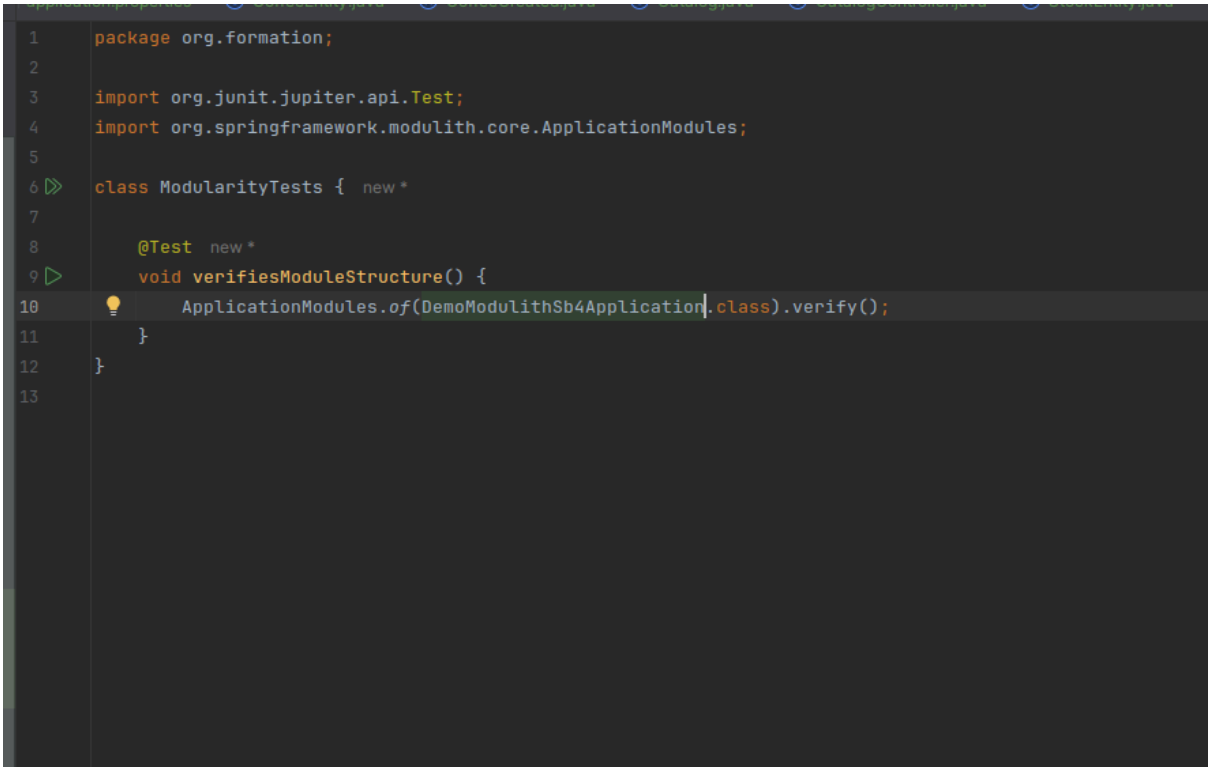
SELECT * FROM COFFEE_ENTITY;

PRICE	ID	NAME
10	1	Espresso

(1 row, 0 ms)

Étape 6 — Vérifier la structure des modules

`src/test/java/org/formation/shop/ModularityTests.java` :



```java
package org.formation;

import org.junit.jupiter.api.Test;
import org.springframework.modulith.core.ApplicationModules;

class ModularityTests {  new *

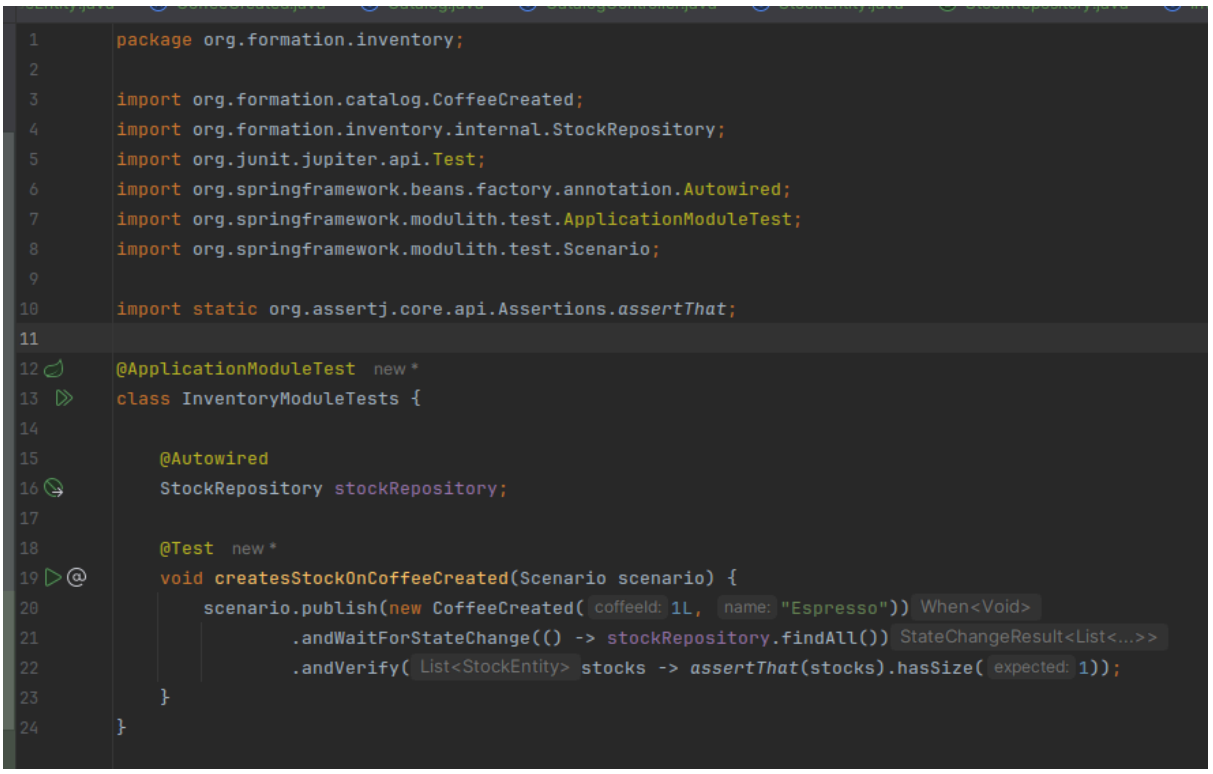
    @Test  new *
    void verifiesModuleStructure() {
        ApplicationModules.of(DemoModulithSb4Application.class).verify();
    }
}
```

ajouter dans `InventoryListener` un accès direct à `CoffeeRepository` (l'`internal` de `catalog`). Le projet compile, mais `verify()` échoue en signalant une dépendance interdite vers un type interne d'un autre module. Retirer l'accès : le test repasse au vert. C'est tout l'intérêt de Modulith.

Étape 7 — Tester un module isolé

`@ApplicationModuleTest` ne démarre que le module visé. L'API `Scenario` publie un événement et attend le changement d'état.

`src/test/java/org/formation/shop/inventory/InventoryModuleTests.java` :



```java
package org.formation.inventory;

import org.formation.catalog.CoffeeCreated;
import org.formation.inventory.internal.StockRepository;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.modulith.test.ApplicationModuleTest;
import org.springframework.modulith.test.Scenario;

import static org.assertj.core.api.Assertions.assertThat;

@ApplicationModuleTest  new *
class InventoryModuleTests {

    @Autowired
    StockRepository stockRepository;

    @Test  new *
    void createsStockOnCoffeeCreated(Scenario scenario) {
        scenario.publish(new CoffeeCreated( coffeeId: 1L,  name: "Espresso")) When<Void>
                .andWaitForStateChange(() -> stockRepository.findAll()) StateChangeResult<List<...>>
                .andVerify( List<StockEntity> stocks -> assertThat(stocks).hasSize( expected: 1));
    }
}
```

`andWaitForStateChange` interroge le dépôt jusqu'à ce que la liste devienne non vide, puis `andVerify` contrôle le résultat. Ce test fonctionne que le listener soit synchrone ou asynchrone, car il attend l'effet.

Note sur le registre et le rejeu

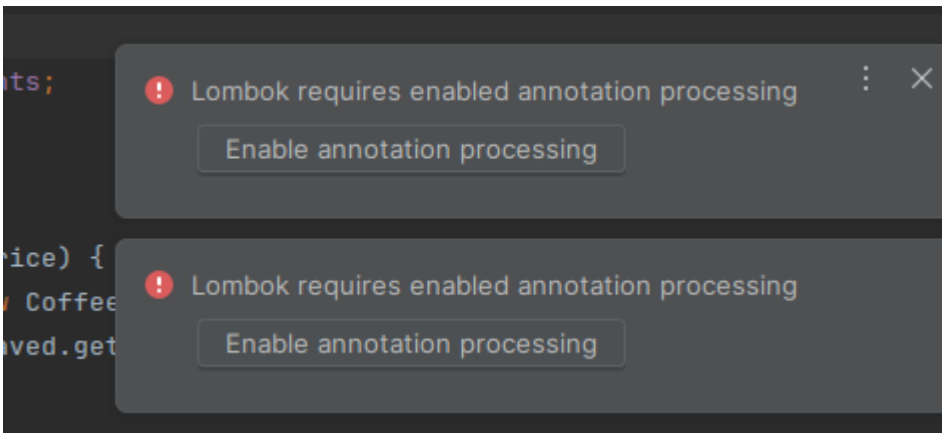
Quand un café est créé, le registre écrit une entrée de publication dans la transaction d'origine, puis la marque comme traitée si le listener réussit. Si le listener échoue, l'entrée reste, et la propriété `spring.modulith.events.republish-outstanding-events-on-restart=true` permet de rejouer les publicatoins en attente au redémarrage. Pour le démontrer, on peut faire échouer volontairement le listener une fois ; c'est une extension facultative, hors du minimum.

À savoir : dès que Modulith est sur le classpath, il persiste tous les listeners `@TransactionalEventListener`, pas seulement les `@ApplicationModuleListener`. Le store JPA (`spring-modulith-starter-jpa`) est donc nécessaire.

Validation

- Un POST sur `/coffees` crée un café et, par événement, une ligne de stock.
- `verify()` passe, et échoue si l'on viole une frontière de module.
- Le test de module crée le stock à la réception de l'événement.

penser à cliquer



Questions

- Pourquoi `catalog` ne dépend-il pas de `inventory` ?
- Que change `@EnableAsync` sur le comportement du listener ?
- À quoi sert l'Event Publication Registry si le listener échoue ?
- En quoi cette organisation prépare-t-elle l'extraction future d'un module en microservice ?